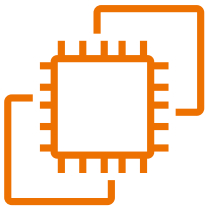

Amazon EC2 — Complete Notes

Elastic Compute Cloud · Resizable virtual servers on AWS

A reader-friendly, all-in-one reference for Amazon EC2.

---

1. 🖥️ What is EC2

Amazon EC2 (**Elastic Compute Cloud**) provides **resizable virtual servers** ("instances") in the cloud. You launch a machine with a chosen OS, CPU, memory, storage, and network capacity in minutes, pay only while it runs, and scale up/down or in/out on demand.

> 💡 **Mental model:** EC2 is **Infrastructure as a Service (IaaS)** — AWS manages the physical host, hypervisor, and data center; **you** manage the OS, patches, runtime, and your application.

Why use it: full control of the OS, any software stack, predictable VM model, lift-and-shift of existing apps, and a huge range of instance sizes (from tiny burstable to GPU/HPC monsters).

Common uses: web/app servers, batch processing, dev/test, databases, game servers, ML training/inference, HPC, hosting almost anything you'd run on a physical server.

---

2. 🧩 Core Concepts

Concept	Description
Instance	A running virtual machine.
AMI	Amazon Machine Image — the template (OS + config + software) an instance boots from.
Instance Type	The hardware profile: vCPUs, memory, network, storage (e.g. `t3.micro`, `m6i.large`).
Region / AZ	Instances run in an Availability Zone within a Region.
Key Pair	Public/private SSH key for login (private key is yours; never re-downloadable).
Security Group	Stateful virtual firewall attached to the instance's network interface.
EBS Volume	Network-attached persistent block storage (virtual disk).
Elastic IP	A static public IPv4 you own and can remap between instances.
ENI	Elastic Network Interface — a virtual NIC (IP, MAC, security groups).
User Data	A script run at first boot to bootstrap the instance.
Tags	Key-value labels for organization, cost allocation, automation.

3. ⚙️ Instance Types & Families

Instance types are grouped into **families** optimized for different workloads. Naming: `m6i.large` → `m` family, `6` generation, `i` Intel (`a` =AMD, `g` =Graviton/ARM), `large` size.

Family	Optimized for	Examples	Use cases
General Purpose	Balanced CPU/mem	`t3`, `t4g`, `m6i`, `m7g`	Web servers, small DBs, dev/test
Compute Optimized	High CPU	`c6i`, `c7g`	Batch, gaming, HPC, media encoding
Memory Optimized	High RAM	`r6i`, `x2idn`, `u-` (high mem)	In-memory DBs, caches, big data, SAP HANA
Storage Optimized	High local disk IOPS/throughput	`i4i`, `d3`, `im4gn`	NoSQL, data warehouses, log processing
Accelerated Computing	GPUs / FPGAs / accelerators	`p5`, `g5`, `inf2`, `trn1`	ML training/inference, graphics, HPC
HPC Optimized	Tightly-coupled HPC	`hpc7g`, `hpc6id`	Simulations, weather, genomics

> **T-family (burstable):** `t2` / `t3` / `t4g` accumulate **CPU credits** when idle and spend them to burst. Great for spiky low-average workloads; watch out for credit exhaustion (or use *unlimited* mode). **Graviton (`g` suffix):** AWS ARM CPUs — better price/performance for many workloads.

4. 💿 AMI (Amazon Machine Image)

An AMI is the **template** an instance boots from: the root volume image (OS + software), launch permissions, and a block-device mapping.

- **Sources:** AWS-provided (Amazon Linux, Ubuntu, Windows…), AWS Marketplace, community AMIs, or **your own custom AMI**.
- **Custom AMIs** bake in your software/config → faster, repeatable launches (golden image).
- AMIs are **region-scoped** — copy an AMI to another region to use it there.
- Can be **private, shared with specific accounts, or public**.
- Backed by **EBS snapshots** (most common) or instance store.

> 🛠️ Typical flow: launch instance → configure it → **Create Image** → launch many identical instances from that AMI (often via a Launch Template + Auto Scaling).

5. 💵 Purchasing / Pricing Options

Pick the model by commitment and tolerance for interruption.

Option	How it works	Savings	Best for
On-Demand	Pay per second/hour, no commitment	Baseline (most expensive)	Spiky / short-term / unpredictable workloads
Savings Plans	Commit to $/hour for 1 or 3 yrs	Up to ~72%	Steady compute usage, flexible across types (Compute SP)
Reserved Instances (RI)	Commit to a specific type/region 1–3 yrs	Up to ~72%	Steady-state, known instance shape
Spot Instances	Bid on spare capacity, can be reclaimed with 2-min notice	Up to ~90%	Fault-tolerant, stateless, batch, CI, big data
Dedicated Hosts	A whole physical server for you	—	Licensing (BYOL), compliance, socket-based licenses
Dedicated Instances	Hardware isolated at the account level	—	Isolation without managing the host
Capacity Reservations	Reserve capacity in an AZ, no term commitment	—	Guaranteeing capacity (e.g. for failover/events)

> 💡 **Spot tips:** use Spot for interruptible work, handle the 2-minute interruption notice, and mix with On-Demand via **Auto Scaling mixed instances policy** or **EC2 Fleet / Spot Fleet**.

6. 💾 Storage Options

Storage	Type	Persists?	Notes
EBS (Elastic Block Store)	Network block volume	✅ Yes (independent of instance)	Most common root + data disk. Snapshot to S3. One AZ.
Instance Store	Local physical disk on host	❌ No (lost on stop/terminate)	Very fast/ephemeral; for caches, scratch, temp data.
EFS (Elastic File System)	Managed NFS (multi-AZ)	✅ Yes	Shared POSIX file system across many instances (Linux).
FSx	Managed file systems	✅ Yes	Windows File Server, Lustre (HPC), NetApp ONTAP, OpenZFS.
S3	Object storage	✅ Yes	Not a disk; accessed via API for backups, data, artifacts.

EBS volume types:

Type	Class	Use case
gp3 / gp2	General-purpose SSD	Default; boot volumes, most workloads (gp3 = baseline 3,000 IOPS, independent throughput)
io2 Block Express / io1	Provisioned IOPS SSD	High-performance DBs, low-latency, high IOPS
st1	Throughput-optimized HDD	Big data, log processing, streaming (sequential)
sc1	Cold HDD	Infrequent access, lowest cost

> EBS volumes are **AZ-scoped**; **snapshots** (stored in S3, regional) let you back up, copy across AZs/regions, and create AMIs.

---

7. 🌐 Networking

- **VPC** — your isolated virtual network. Instances launch into a **subnet** within an AZ.
- **Public subnet** → has a route to an **Internet Gateway (IGW)**. **Private subnet** → reaches the internet only via a **NAT Gateway**.
- **Private IP** — internal, stays with the instance for its life. **Public IP** — internet-facing, **changes on stop/start** (dynamic).
- **Elastic IP (EIP)** — a **static** public IPv4 you own; remap it between instances. (Charged when allocated but not in use.)
- **ENI** — virtual network card; an instance can have multiple ENIs / multiple IPs.
- **Enhanced Networking (ENA / SR-IOV)** and **placement groups** boost throughput / lower latency.
- **IPv6** supported; **Bandwidth** scales with instance size.

> Connectivity to other networks: **VPC Peering**, **Transit Gateway**, **VPN**, **Direct Connect**, **PrivateLink / VPC endpoints**.

---

8. 🛡️ Security Groups & NACLs

A **Security Group** is a **virtual, stateful firewall** attached to an instance's network interface (ENI). It controls traffic with **allow rules** that match a **protocol + port + source/destination** (an IP/CIDR or another security group). Because it's **stateful**, if you allow a request in, the response is automatically allowed back out — you don't write a return rule. By default a security group **denies all inbound** and **allows all outbound**, and an instance can have several attached at once (their rules are combined). It's your primary, instance-level control; the NACL below is the coarser subnet-level backstop.

Two layers of network filtering — know the difference:

	Security Group	Network ACL (NACL)
Scope	Instance / ENI level	Subnet level
State	**Stateful** (return traffic auto-allowed)	**Stateless** (must allow both directions)
Rules	**Allow only**	Allow **and** Deny
Evaluation	All rules evaluated	Rules in **numbered order**, first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows all; custom denies all

> ✅ **Best practice:** reference **other security groups** as sources (not raw IPs) for tier-to-tier rules; open only the ports you need (e.g. 22/SSH, 443/HTTPS) and restrict source CIDRs.

---

9. 🔑 Key Pairs & IAM Roles

Key Pairs (SSH/RDP login):

- AWS stores the **public** key; you keep the **private** key (`.pem`). It is shown **only once at creation** — losing it means you can't SSH in (recover via other methods).
- Linux: SSH with the private key (`ssh -i key.pem ec2-user@ip`). Windows: decrypt the admin password with the private key for RDP.

IAM Roles / Instance Profiles:

- Attach an **IAM role** to an instance so apps on it get **temporary credentials** automatically (via the metadata service) — **never hard-code access keys** on an instance.
- The role defines what AWS APIs the instance may call (e.g. read an S3 bucket, write CloudWatch logs).

---

10. 📜 User Data & Instance Metadata

User Data

- A script (bash / PowerShell / cloud-init) passed at launch and run **once on first boot** to bootstrap (install packages, configure, pull code).
- Can be set to run on every boot with extra config.

Instance Metadata Service (IMDS)

- Reachable from inside the instance at `http://169.254.169.254/latest/meta-data/` — exposes instance id, AMI id, IP, IAM role credentials, user data, etc.
- **Use IMDSv2** (session/token-based) — more secure against SSRF than the legacy IMDSv1. Enforce it (`HttpTokens=required`).

```
# Example: fetch metadata using IMDSv2
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
  -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
  http://169.254.169.254/latest/meta-data/instance-id
```

---

11. 📍 Placement Groups

Control how instances are physically placed to balance performance vs. resilience:

Strategy	Placement	Use for
Cluster	Packed close in one AZ	Low latency, high throughput (HPC, tight node-to-node)
Spread	Spread across distinct hardware/racks	Critical instances that must not share failure domains
Partition	Grouped into isolated partitions	Large distributed/replicated systems (HDFS, Kafka, Cassandra)

---

12. 📈 Auto Scaling

EC2 Auto Scaling Group (ASG) automatically adjusts the number of instances to match demand.

- **Launch Template** (preferred over launch config) defines *what* to launch (AMI, type, key, SGs, user data).
- **Min / Desired / Max** capacity bounds the group.
- **Scaling policies:** Target Tracking (e.g. keep CPU at 50%), Step scaling, Simple scaling, Scheduled scaling, Predictive scaling.
- **Health checks** (EC2 or ELB) replace unhealthy instances automatically → **self-healing**.
- Spread instances across **multiple AZs** for high availability.
- Integrates with **ELB** to register/deregister instances automatically.

> 💡 Auto Scaling = elasticity (right-size to load) **and** resilience (auto-replace failed nodes). It does **not** cost extra — you pay only for the instances.

---

13. ⚖️ Load Balancing

Elastic Load Balancing (ELB) distributes traffic across instances/targets in multiple AZs.

Type	Layer	Best for
ALB (Application LB)	L7 (HTTP/HTTPS)	Web apps, path/host routing, microservices, containers
NLB (Network LB)	L4 (TCP/UDP/TLS)	Ultra-high performance, static IP, low latency
GWLB (Gateway LB)	L3	Deploying virtual appliances (firewalls, IDS/IPS)
CLB (Classic, legacy)	L4/L7	Older accounts only — avoid for new work

> Pair ELB + Auto Scaling across AZs for a highly available, self-healing tier. Use **health checks** so traffic only goes to healthy targets.

---

14. 🔄 Instance Lifecycle

States: `pending → running → stopping → stopped → shutting-down → terminated` (also `rebooting`).

Action	What happens	Data
Stop	Shuts down; EBS root persists; public IP released	EBS data kept; instance-store lost
Start	Boots again (maybe new host, new public IP)	EBS data intact
Reboot	Restart in place; stays on same host	Everything kept (incl. instance store)
Hibernate	RAM saved to EBS, resumes state on start	Memory preserved (size/AMI limits apply)
Terminate	Instance deleted	Root EBS deleted by default (unless "delete on termination" off); attached data volumes per setting

> ⚠️ **Stop ≠ Terminate.** Terminate is permanent. Enable **termination protection** for important instances. Instance-store data is **always lost** on stop/terminate.

---

15. 📊 Monitoring & Status Checks

- **CloudWatch metrics** (default, 5-min; **detailed** = 1-min, extra cost): CPU, network, disk I/O, status checks. *Note: memory & disk usage need the **CloudWatch agent**.*
- **Status checks:**
  - **System status check** — AWS infrastructure (host, network, power). Fix = stop/start (moves to new host).
  - **Instance status check** — the instance/OS itself (bad config, exhausted memory, kernel). Fix = reboot / fix OS.
- **CloudWatch Alarms** → trigger actions (notify, auto-recover, scale, stop/terminate).
- **EC2 Auto Recovery** for failed system checks; **CloudWatch Logs** + agent for app/OS logs.

- **CloudTrail** logs EC2 API calls for audit.

---

16. 💰 Pricing

You pay for:

1. **Instance compute** — per second (min 60s) or hour, by type & purchase option.
2. **EBS storage** — per GB-month provisioned, plus IOPS/throughput for io/gp3, plus snapshots.
3. **Data transfer OUT** — to internet *(IN is free; cross-AZ and inter-region transfer is charged)*.
4. **Elastic IPs** — free while attached to a running instance; charged when allocated and idle.
5. **Extras** — detailed monitoring, ELB hours + LCUs, NAT Gateway, etc.

> 💸 **Cost tips:** right-size instances · use **Graviton** for price/performance · **Spot** for interruptible work · **Savings Plans/RIs** for steady usage · **stop** non-prod overnight · delete unused EBS volumes + old snapshots · release idle EIPs · use **Compute Optimizer** recommendations.

---

17. ⌨️ Common CLI Commands

```
# List instances (id, type, state)
aws ec2 describe-instances \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType,State.Name]' --output table

aws ec2 run-instances --image-id ami-123 --instance-type t3.micro \
  --key-name mykey --security-group-ids sg-123 --subnet-id subnet-123  # launch

aws ec2 stop-instances      --instance-ids i-123     # stop
aws ec2 start-instances     --instance-ids i-123     # start
aws ec2 reboot-instances    --instance-ids i-123     # reboot
aws ec2 terminate-instances --instance-ids i-123     # terminate

aws ec2 describe-instance-status --instance-ids i-123   # status checks
aws ec2 create-image --instance-id i-123 --name "my-ami"  # bake an AMI
aws ec2 describe-security-groups                          # list SGs
aws ec2 allocate-address                                  # get an Elastic IP

# Connect without SSH keys (needs SSM agent + IAM role)
aws ssm start-session --target i-123
```

---

18. 📌 Key Limits & Numbers

Item	Value
Billing granularity	Per-second (60s minimum) for Linux; per-hour for some
Spot interruption notice	**2 minutes**
Default running On-Demand limit	vCPU-based quota per family (adjustable)
Security groups per ENI	5 (default, up to 16 on request)
Rules per security group	60 inbound + 60 outbound (default)
EBS volume size	1 GiB – 64 TiB (per volume)
Max IOPS (io2 Block Express)	up to 256,000
Elastic IPs per region	5 (default, adjustable)
Metadata endpoint	`169.254.169.254` (link-local)
EBS snapshots	Stored in S3, incremental, regional

19. 🔌 Ways to Connect to EC2

Method	How	Best for
SSH	`ssh -i key.pem ec2-user@<public-ip>` (Linux)	Direct Linux access with a key pair + port 22 open
RDP	Decrypt admin password with private key, connect via RDP client (port 3389)	Windows instances
EC2 Instance Connect	Browser/CLI push of a temporary SSH key (Amazon Linux/Ubuntu)	Quick keyless-feeling SSH without managing keys
SSM Session Manager	`aws ssm start-session --target i-123`	**No open inbound ports, no key pair** — needs SSM agent + IAM role. Most secure; auditable.
Serial Console	Browser serial connection	Troubleshooting boot/network issues
Bastion / Jump host	SSH through a public bastion to private instances	Reaching instances in private subnets

> 🔐 **Best practice:** prefer **SSM Session Manager** — it removes the need for open SSH ports, bastions, and long-lived keys, and logs every session. Default OS users: Amazon Linux `ec2-user`, Ubuntu `ubuntu`, RHEL `ec2-user` / `root`, Debian `admin`.

🧭 Quick pick guide

Need	Use
Quick Linux shell, key in hand	SSH
Windows desktop	RDP
No keys / no open ports / audited	SSM Session Manager
One-off SSH without storing keys	EC2 Instance Connect
Instance won't boot / network broken	Serial Console
Private-subnet instance	Bastion host or SSM

---

20. 🧠 Quick Mental Model

- **EC2 = rentable virtual servers.** You manage the OS and up; AWS manages the hardware.
- Choose an **AMI** (the image) + an **instance type** (the hardware) + a **subnet** (the network).
- **Security Groups** are your stateful firewall; open only what you need.
- Use **EBS** for persistent disks, **snapshots** for backups, **instance store** only for scratch.
- Pick a **pricing model**: On-Demand (flexible) · Spot (cheap + interruptible) · Savings Plans/RIs (steady).
- **Auto Scaling + ELB across AZs** = elastic, self-healing, highly available.
- Give instances an **IAM role**, never hard-coded keys; use **IMDSv2**.
- Prefer **SSM Session Manager** to connect — no open ports, no key sprawl.
- **Monitor** with CloudWatch + status checks; **right-size** relentlessly to control cost.

---

21. ❓ Common Interview Questions

Rapid-fire questions interviewers ask about EC2:

- **Q: On-Demand vs Reserved vs Spot?** — On-Demand = pay-as-you-go, no commitment. Reserved/Savings Plans = 1–3 yr commitment, big discount, steady workloads. Spot = spare capacity up to ~90% off, can be reclaimed with a 2-minute notice → fault-tolerant work only.
- **Q: Stop vs Terminate vs Reboot?** — Stop keeps the EBS root (new public IP on start); Terminate deletes the instance (root EBS deleted by default); Reboot restarts in place keeping everything.
- **Q: Security Group vs NACL?** — SG = stateful, instance-level, allow-only. NACL = stateless, subnet-level, allow + deny in numbered order.
- **Q: How do apps on EC2 access AWS securely?** — Attach an **IAM role** (instance profile) → temporary auto-rotated credentials. Never hard-code access keys.
- **Q: EBS vs Instance Store?** — EBS = network block storage that persists independent of the instance. Instance store = ephemeral local disk, lost on stop/terminate.
- **Q: How do you make an app highly available?** — Auto Scaling Group across multiple AZs behind an ELB, with health checks for self-healing.

- **Q: What's a burstable (T-family) instance?** — Accrues CPU credits when idle and spends them to burst; good for spiky low-average workloads.
- **Q: IMDSv1 vs IMDSv2?** — IMDSv2 is token/session-based, hardened against SSRF; enforce `HttpTokens=required`.

---

22. 🎯 Detailed Interview Q&A and When to Use

> Scenario-style interview questions with model answers and **when to use** guidance. Aimed at ~2 years of hands-on experience. Each answer leads with the core point, then the trade-off an interviewer probes.

Basics

1. What is EC2, and why not run your own servers? Resizable virtual compute on demand. You trade capex for opex, get elasticity (scale in minutes), and offload data-center / hardware / networking heavy lifting to AWS. You still own the OS upward. **When to use:** EC2 → need OS-level control, long-running/stateful, custom agents or kernels, lift-and-shift. Fargate/ECS/EKS → containerized and you don't want to manage instances. Lambda → short, event-driven, spiky, < 15 min.

2. Instance families (t / m / c / r)? `t` = burstable, cheap. `m` = balanced. `c` = compute-optimized (high CPU:RAM). `r` = memory-optimized. Pick `c` over `m` when CPU-bound, not RAM-bound. **When to use:** `t` → dev, low/spiky traffic, cost-sensitive. `m` → general web/app tier. `c` → batch, encoding, gaming, ML inference. `r` → Redis/Memcached, in-memory DBs, big JVMs.

3. Stop vs Terminate vs Hibernate? Stop: halts, root EBS persists, public IP released, compute billing stops. Terminate: gone, root EBS deleted by default. Hibernate: RAM dumped to root EBS, resumes exact state. Instance-store data survives only reboot — lost on stop and terminate. **When to use:** Stop → pause to save cost, restarting soon, OK to lose RAM and get a new public IP. Hibernate → restart must resume exact RAM state (long warm-up, in-memory caches, slow-booting licensed apps). Terminate → done permanently; let the root volume delete.

4. AMI vs snapshot? AMI = bootable template (root volume + block-device mapping + launch metadata) used to launch instances. Snapshot = point-in-time backup of a single EBS volume. An AMI references one or more snapshots; a snapshot alone can't boot. **When to use:** AMI → standardize/launch identical instances (golden image, ASG, faster boot). Snapshot → back up/restore or migrate a single volume; copy across region/account.

5. Security Group vs NACL? SG = instance-level (ENI), stateful, allow-only. NACL = subnet-level, stateless, allow + explicit deny, evaluated by rule number. SGs are the primary control; NACLs are a coarse subnet backstop. **When to use:** SG → your everyday instance firewall (always). NACL → subnet-wide coarse rules, explicit deny (block an IP range), compliance segmentation.

6. SG "stateful" meaning? Return traffic for an allowed inbound connection is automatically permitted outbound (and vice versa) — no reverse rule needed. NACLs are stateless, so you must open both directions (and ephemeral ports for replies).

7. Purchasing options + use case? On-Demand = no commitment. Reserved = 1/3-yr commit to instance type. Savings Plans = commit to $/hr spend, flexible across families. Spot = spare capacity up to ~90% off, interruptible. **When to use:** On-Demand → unpredictable/short-term, dev. Reserved → steady 24/7 baseline, known type. Savings Plans → steady spend but want family flexibility. Spot → fault-tolerant, interruptible (batch, CI, stateless web behind ASG).

Intermediate

8. Instance store vs EBS — and when instance store? Instance store = physically attached, ephemeral, very high IOPS, dies on stop/terminate. EBS = network-attached, durable, persists independently. **When to use:** Instance store → scratch/cache/temp, ultra-high IOPS, data you can lose (local shuffle, buffers). EBS → anything that must persist past stop/terminate (root, DB data).

9. EBS types gp3 / gp2 / io2 / st1? gp3 = default SSD, IOPS/throughput decoupled from size. gp2 = older SSD, IOPS scales with size. io2 = provisioned-IOPS SSD for critical low-latency DBs. st1 = throughput HDD for big sequential. gp3 won because it's cheaper and you provision IOPS independently. **When to use:** gp3 → default for almost everything. gp2 → legacy only. io2 (Block

Express) → mission-critical low-latency DBs needing guaranteed high IOPS. st1 → big sequential throughput (logs, data lake, streaming).

10. gp3 decoupled IOPS — why it matters for cost? On gp2 you over-sized the disk just to buy IOPS. On gp3 you size storage for capacity and dial IOPS/throughput separately — no paying for terabytes you don't need to hit a performance target.

11. Public IP vs Elastic IP vs private IP? Private IP = stable, internal to VPC. Public IP = auto-assigned, changes on stop/start, lost on stop. EIP = static public IP you own, survives stop/start. **When to use:** EIP → an external system needs a fixed public IP across stop/start (whitelisted firewall, DNS A-record to instance). Otherwise prefer a load balancer / DNS.

12. Private subnet → internet for OS updates? Instance → route table default route → NAT Gateway in a public subnet → Internet Gateway → internet. Return traffic comes back through the NAT. Outbound-initiated only; no inbound exposure.

13. NAT Gateway vs NAT Instance — why ever a NAT instance? NAT GW is managed, HA-per-AZ, auto-scaling. NAT instance is a self-managed EC2. **When to use:** NAT Gateway → default. NAT instance → tiny/cost-sensitive, or you need port-forwarding / custom firewall / bastion combo.

14. User data — reboot vs first-boot gotcha? User data runs once at first boot by default. On reboot/stop-start it does not re-run unless cloud-init is configured (`cloud-init-per always`) to do so. **When to use:** Run-once (default) → one-time bootstrap. cloud-init `always` → re-pull config/secrets or re-register on every boot.

15. IAM roles for EC2 vs baked access keys? The instance assumes a role and gets temporary, auto-rotated credentials via the metadata service. No long-lived keys on disk to leak; centrally managed and revocable. **When to use:** Instance role → always, for AWS API access. Static access keys → only when something outside AWS genuinely can't assume a role.

16. Launch Template vs Launch Configuration? Launch Template = versioned, supports newer features (multiple instance types, mixed Spot/On-Demand, T-unlimited). Launch Configuration = older, immutable, no versioning — deprecated. **When to use:** Launch Template → always. Launch Configuration → never for new work.

Advanced & Scenario

17. Scenario — DB unreachable after stop/start, no code change. Top hypotheses: (a) public IP changed and something referenced it; (b) the DB's security group references the instance's old IP/EIP; (c) lost instance-store data or a mount. Confirm: compare new public/private IP, check SG rules referencing IPs, route table, `nc -vz db 5432` , DNS resolution.

18. Scenario — SSH hangs / refused, full triage. Instance running and status checks passing (instance vs system check)? Network path: SG inbound 22 from your IP, NACL, route to IGW, correct public IP/subnet. `nc -vz host 22` — refused = sshd down/wrong port; timeout = network/SG/NACL. Then OS: disk-full/high-CPU can hang sshd — use SSM Session Manager or EC2 Serial Console to get in without SSH.

19. Scenario — ASG launches instances that fail health checks and loop. Instances die before you can SSH, so capture state at boot: ship cloud-init/user-data logs to CloudWatch, check ASG activity history and health-check type. Temporarily raise the health-check grace period or suspend `ReplaceUnhealthy` / `Terminate` so one stays up. Usual causes: app not listening on the health-check port/path, missing IAM/role, bad AMI, grace period too short.

20. Placement groups (cluster / spread / partition)? Cluster = same rack, low-latency/high-throughput. Spread = each instance on distinct hardware, max fault isolation. Partition = grouped racks isolated from each other. **When to use:** Cluster → HPC, tightly-coupled. Spread → small set of critical instances. Partition → large distributed data systems (HDFS, Cassandra, Kafka).

21. ALB health check; ELB vs ASG health-check disagreement? ALB marks a target unhealthy after N failed checks (path/port/codes). The ASG's default EC2 check only tests hypervisor reachability, so a hung-but-running app passes it and never gets replaced. **When to use:** ASG health-check type EC2 → you only care the box is reachable. ELB type → you care the app responds (almost always — replaces hung instances).

22. Scenario — t3 at 100% CPU but barely working? It exhausted its CPU credits and is throttled to baseline. Burstable instances accrue credits when idle and spend them to burst; at zero credits you're capped, so it looks pegged while doing little. Move to non-burstable or t3 unlimited.

23. CPU credits; t3 standard vs unlimited; cost trap? Standard bursts only while credits last, then throttles. Unlimited bursts beyond credits and bills surplus as overage. **When to use:** Standard → predictable low baseline, OK to throttle. Unlimited →

occasional bursts where throttling is unacceptable — but if persistently above baseline, a larger non-burstable instance is cheaper than overage.

24. Scenario — stateful, no-interruption app, but Finance wants 60% savings. Spot alone breaks a no-interruption stateful app. Externalize state (RDS/EBS/S3) so compute is replaceable, run a mixed ASG — On-Demand/Reserved baseline for guaranteed capacity + Spot for burst — and apply Savings Plans to the baseline. Most of the savings without betting the stateful core on Spot reclaim.

25. Why ASG + ALB over 3 fixed instances behind a static IP? For: self-healing, elasticity, rolling deploys, AZ spread, health-based routing. Against: more moving parts, ALB cost, requires statelessness. **When to use:** ASG + ALB → production, variable load, need HA/self-healing/rolling deploys. Fixed instances → tiny, predictable, internal-only, hand-managed.
